

第10章 排序

Faculty of Information and Control Engineering

Huanliang Sun

Email: sunhl@sjzu.edu.cn

Office: Wu(戊2-306) 24690666

第十章 内部排序

第一次课

10.1 概述

10.2 插入排序

练习与作业

第二次课

[10.3 快速排序](#)

[10.4 选择排序](#)

[练习与作业](#)

第三次课

10.5 归并排序

10.6 基数排序

交换排序：借助“交换”进行排序

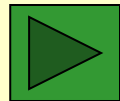
一、冒泡排序

□排序过程：

- 将第一个记录的关键字与第二个记录的关键字进行比较，若为逆序 $L.r[1].key > L.r[2].key$ ，则交换；然后比较第二个记录与第三个记录；依次类推，直至第 $n-1$ 个记录和第 n 个记录比较为止
- 第一趟冒泡排序，结果关键字最大的记录被安置在最后一个记录上。
- 对前 $n-1$ 个记录进行第二趟冒泡排序，结果使关键字次大的记录被安置在第 $n-1$ 个记录位置。
- 重复上述过程，直到“在一趟排序过程中没有进行过交换记录的操作”为止。

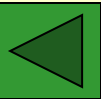
例	38	38	38	38	13	13	13
	49	49	49	13	27	27	27
	65	65	13	27	30	30	30
	76	13	27	30	38	38	
	13	27	30	49	49		
	27	30	65	65			
	30	76	76				
	97	97	第二趟	第三趟	第四趟	第五趟	第六趟
初始关键字	第一趟						

第*i*趟完成时，会有有序的*i*个以上的数出现在一侧位置



算法描述:

```
void BubbleSort(SqList &L){  
    // 对顺序表L作冒泡排序  
    for (i=L.length, change=TRUE; i>1 && change; --i) {  
        change = FALSE;  
        for (j=1; j<i; ++j)  
            if (L.r[j].key > L.r[j+1].key){  
                L.r[j]←→L.r[j+1];  
                change = TRUE; //用于减少比较次数, 早停止  
            }  
    }  
} // for I  
} // BubbleSort
```



算法评价

■ 时间复杂度

➤ 最好情况（正序）

☆ 比较次数: $n-1$

☆ 移动次数: 0

➤ 最坏情况（逆序）

☆ 比较次数: $\sum_{i=1}^{n-1} (n-i) = \frac{1}{2}(n^2 - n)$

☆ 移动次数: $3\sum_{i=1}^n (n-i) = \frac{3}{2}(n^2 - n)$

$$T(n) = O(n^2)$$

■ 空间复杂度: $S(n) = O(1)$

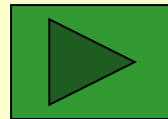
二、快速排序——对起泡排序的改进，分治策略

基本思想：通过一趟排序，将待排序记录分割成独立的两部分，其中一部分记录的关键字均比另一部分记录的关键字小，则可分别对这两部分记录进行排序，以达到整个序列有序

枢轴：中间分界的关键字称枢轴

排序过程：对 $R[s \dots t]$ 中记录进行一趟快速排序，附设两个指针 i 和 j ，设枢轴记录 $rp=R[s]$ ， $x=rp.key$

- 初始时令 $i=s$ ， $j=t$
- 首先从 j 所指位置向前搜索第一个关键字小于 x 的记录，并和 rp 交换
- 再从 i 所指位置起向后搜索，找到第一个关键字大于 x 的记录，和 rp 交换
- 重复上述两步，直至 $i=j$ 为止
- 再分别对两个子序列进行快速排序，直到每个子序列只含有一个记录为止



Example

$i = \text{low}$
 $j = \text{high}$

Original order: $\begin{matrix} x \\ || \end{matrix}$

27	38	13	49	76	97	65	50
↑	↑	↑	↑↑	↑	↑	↑	↑
i	i	i	ij	j	j	j	j

The first partition : (27 38 13) 49 (76 97 65 50)

Further : (13) 27 (38) 49 (50 65) 76 (97)

Result: 13 27 38 49 50 65 76 97

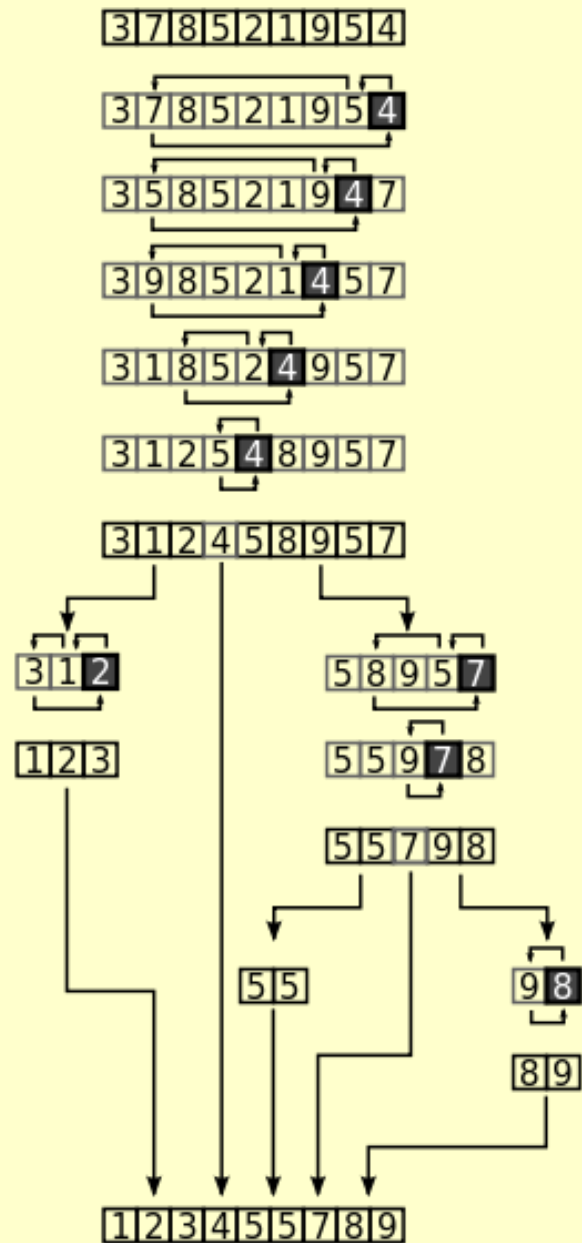
```
int Partition (ElemType *R, int low, int high) {
    pivot = R[low];
    while (low < high) {
        while (low < high && R[high] >= pivot) --high;
        R[low] ↔ R[high];    //pseudo code
        while (low < high && R[low] <= pivot) ++low;
        R[low] ↔ R[high];
    }
    return low;    //return the position of the pivot
} // Partition
```

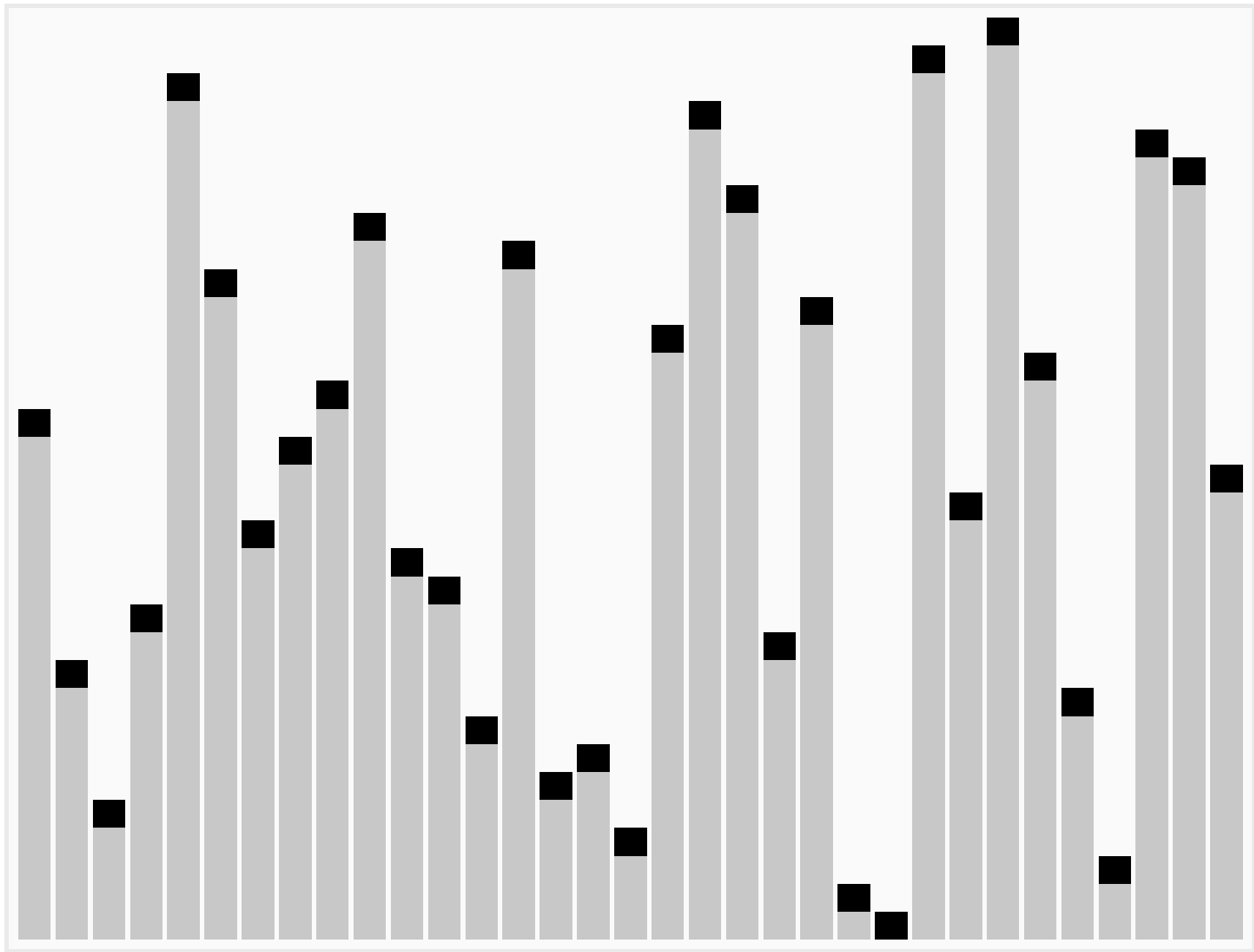
- 为了减少记录的移动次数，应先将枢轴记录“移出”，待求得枢轴记录应在的位置之后（此时 $\text{low} == \text{high}$ ），再将枢轴记录到位。
- 一趟指一次划分，第 i 趟完成时，会有 i 个以上的数出现在它最终将要出现的位置


```
int Partition (ElemType *R, int low, int high)
{
    ElemType pivot;
    pivot = R[low];
    while (low<high) {
        while (low<high && R[high] >=pivot)  --high;
        _____;    //assign the value once
        while (low<high && R[low]<=pivot)  ++low;
        _____;
    }
    R[low] = pivot;
    return low;    //return the position of the pivot
} // Partition
```

Quicksort

```
void QuickSort (ElemType *R, int low, int high)
{
    int q;
    if (low < high) {
        q=Partition(R,low,high);
        QuickSort (____,____,____); //sort the left
        QuickSort (____,____,____); //sort the right
    }
}
```





- **时间效率**：在 n 个记录的待排序列中，一次划分为 $O(n)$ ，若设 $T(n)$ 为对 n 个记录的待排序列进行快速排序所需时间。
- **理想情况**：（每次总是选到中间值作枢轴） 每次划分，正好将分成两个等长的子序列，则 $T(n)=O(n\log_2 n)$
- **最坏情况**：（每次总是选到最小或最大元素作枢轴） 时效为 $O(n^2)$ 。

快速排序是通常被认为在同数量级（ $O(n\log_2 n)$ ）的排序方法中平均性能最好的。若初始序列按关键码有序或基本有序时，快排序反而蜕化为冒泡排序。为改进，通常以“三者取中法”来选取支点记录，即将排序区间的两个端点与中点三个记录关键码居中的调整为支点记录。

非递归要借助栈来实现

2. Picking the Pivot

👉 A Wrong Way: **Pivot = A[0]**

The worst case: A[] is **presorted** – quicksort will take $O(N^2)$ time to do **nothing** 😞

👉 A Safe Maneuver: **Pivot = random select from A[]**

😞 random number generation is **expensive**

👉 Median-of-Three Partitioning:

Pivot = median (left, center, right)

Eliminates the bad case for sorted input and actually reduces the running time by about 5%.

```
/* Return median of low, Center, and high*/  
/* Order these and hide the pivot */
```

```
ElementType Median3( ElementType A[ ], int low, int high )  
{  
    int Center = (low + high ) / 2;  
    if ( A[low ] > A[ Center ] )  
        Swap( &A[low], &A[ Center ] );  
    if ( A[low] > A[high ] )  
        Swap( &A[low], &A[high ] );  
    if ( A[ Center ] > A[ Right ] )  
        Swap( &A[Center ], &A[high] );  
    /* Invariant: A[low ] <= A[Center ] <= A[high ] */  
    Swap( &A[ Center ], &A[high - 1 ] ); /* Hide pivot */  
    /* only need to sort A[low + 1 ] ... A[high - 2 ] */  
    return A[high - 1 ]; /* Return pivot */  
}
```

```

void Qsort( ElementType A[ ], int low, int high )
{
    int i, j;
    ElementType Pivot;
    if (low + Cutoff <= high ) { /* if the sequence is not too short */
        Pivot = Median3( A, low, high ); /* select pivot */
        i = low;    j = high - 1; /* why not set Left+1 and Right-2? */
        while(1) {
            while ( A[ ++i ] < Pivot ) { } /* scan from low */
            while ( A[ --j ] > Pivot ) { } /* scan from right */
            if ( i < j )
                Swap( &A[ i ], &A[ j ] ); /* adjust partition */
            else    break; /* partition done */
        }
        Swap( &A[ i ], &A[ high - 1 ] ); /* restore pivot */
        Qsort( A, low, i - 1 ); /* recursively sort low part */
        Qsort( A, i + 1, high ); /* recursively sort high part */
    } /* end if - the sequence is long */
    else /* do an insertion sort on the short subarray */
        InsertionSort( A + low, high - low + 1 );
}

```


Experiment 10-2 Quicksort

一、简单选择排序

□ 排序过程

- 首先通过 $n-1$ 次关键字比较，从 n 个记录中找出关键字最小的记录，将它与第一个记录交换
- 再通过 $n-2$ 次比较，从剩余的 $n-1$ 个记录中找出关键字次小的记录，将它与第二个记录交换
- 重复上述操作，共进行 $n-1$ 趟排序后，排序结束

例 i=1 初始: [13 38 65 97 76 49 27]

Diagram showing initial state with pointers k and j:

```

      k       k
      ↓       ↓
[ 13 38 65 97 76 49 27 ]
      ↑       ↑
      j       j
  
```

i=2 一趟: 13 [27 65 97 76 49 38]

Diagram showing first pass with pointers k and j:

```

      k
      |
13 [ 27 65 97 76 49 38 ]
      ↑
      j
  
```

二趟: 13 27 [65 97 76 49 38]

Diagram showing second pass with pointers k and j:

```

      k
      |
13 27 [65 97 76 49 38]
      ↑
      j
  
```

三趟: 13 27 38 [97 76 49 65]

Diagram showing third pass with pointers k and j:

```

      k
      |
13 27 38 [97 76 49 65]
      ↑
      j
  
```

四趟: 13 27 38 49 [76 97 65]

Diagram showing fourth pass with pointers k and j:

```

      k
      |
13 27 38 49 [76 97 65]
      ↑
      j
  
```

五趟: 13 27 38 49 65 [97 76]

Diagram showing fifth pass with pointers k and j:

```

      k
      |
13 27 38 49 65 [97 76]
      ↑
      j
  
```

六趟: 13 27 38 49 65 76 [97]

Diagram showing sixth pass with pointers k and j:

```

      k
      |
13 27 38 49 65 76 [97]
      ↑
      j
  
```

排序结束: 13 27 38 49 65 76 97

第i趟完成时，会有有序的i个以上的数出现在一侧位置

	8
	5
	2
	6
	9
	3
	1
	4
	0
	7

算法描述：

```
void SelectSort (SqList &L) {  
    // 对顺序表L作简单选择排序  
    for (i=1; i<L.length; ++i) {  
        // 选择第 i 小的记录，并交换到位  
        j = i;  
        for ( k=i+1; k<=L.length; k++ )  
            // 在L.r[i..L.length]中选择关键字最小的记录  
            if ( L.r[k].key < L.r[j].key )  
                j = k ;  
        if ( i!=j ) L.r[j]  $\longleftrightarrow$  L.r[i];  
        // 与第 i 个记录互换  
    } // for  
} // SelectSort
```

```
Void selectionSort(int a[ ]){  
    /* a[0] to a[n-1] is the array to sort */  
    int i, j, iMin;  
    /* advance the position through the entire array */  
    /* (could do j < n-1 because single element is also min element) */  
    for (i = 0; i < n-1; i++) { /* find the min element in the unsorted a[j .. n-1] */  
        iMin = i; /* assume the min is the first element */  
        for ( j = i+1; j < n; j++) { /* test against elements after j to find the  
smallest */  
            /* if this element is less, then it is the new minimum */  
            if (a[j] < a[iMin]) {  
                iMin = j; /* found new minimum; remember its index */  
            }  
        }  
        if(iMin != i) {  
            swap(&a[i], &a[iMin]);  
        }  
    }  
}
```

```
#include <stdio.h> /* a[0] to a[n-1] is the array to sort */
```

```
#define ARR_SIZE 30
```

```
void selectionSort (float a[], int n);
```

```
void swap(float *x, float *y);
```

```
main( )
```

```
{
```

```
    int k, n;
```

```
    float a[ARR_SIZE]={0.0};
```

```
    printf("Input the number of score:");
```

```
    scanf(" %d", &n);
```

```
    printf("Input %d score:", n);
```

```
    for (k=0; k<n; k++)
```

```
        scanf("%f", &a[k]);
```

```
    printf("The original order is:\n");
```

```
    for (k=0; k<n; k++)
```

```
        printf("%f\n", a[k]);
```

```
selectionSort (a, n);
```

```
    printf("The new order is:\n");
```

```
    for (k=0; k<n; k++)
```

```
        printf("%f\n", a[k]);
```

```
    system("PAUSE");
```

```
}
```

```
void swap(float *x, float *y)
```

```
{
```

```
    float temp;
```

```
    temp = *x;
```

```
    *x = *y;
```

```
    *y = temp;
```

```
}
```

■ 算法评价

➤ 时间复杂度

■ 记录移动次数

☆ 最好情况：0

☆ 最坏情况：3(n-1)

■ 比较次数：

$$\sum_{i=1}^{n-1} (n-i) = \frac{1}{2} (n^2 - n)$$

$$T(n) = O(n^2)$$

➤ 空间复杂度：S(n)=O(1)

Experiment 10-1 Selection Sorting for strings

Problems:

String Sorting by selection sorting algorithm.
(Given selection sorting for int values)

For example: To sort students by names.

//char arr[row][col] 函数只认为这是一个二维的字符数组，不会把每个一维数组作为字符指针（也就是字符串）去处理

//char * ptr[row] 函数会认为这是一个一维数组，每个元素是一个字符指针。

```
strcmp(str[iMin], str[j])>0
```

```
//swap the pointers
```

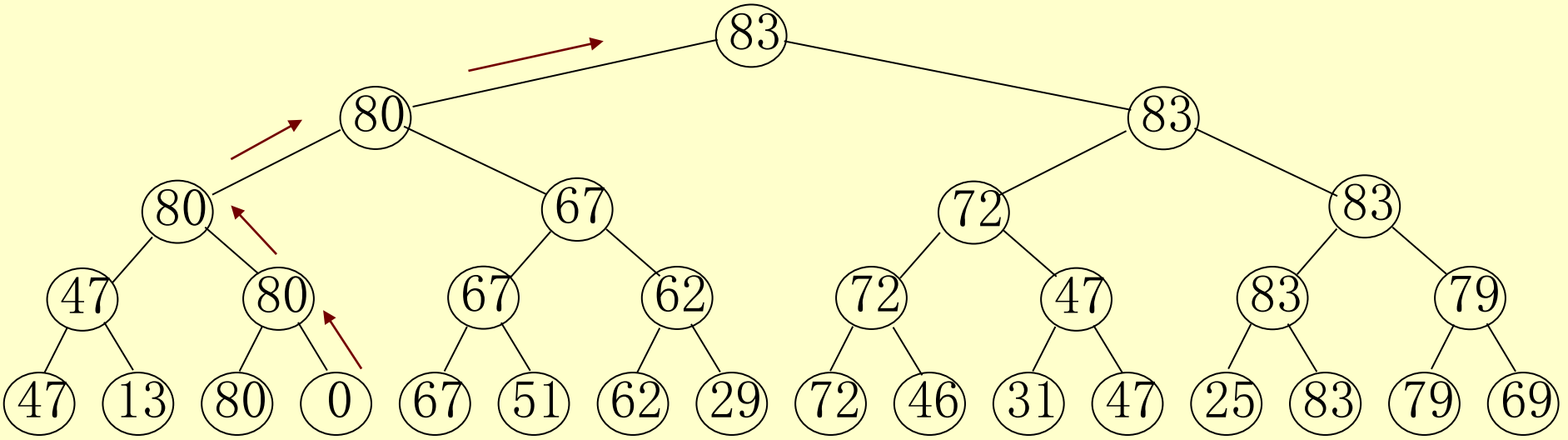
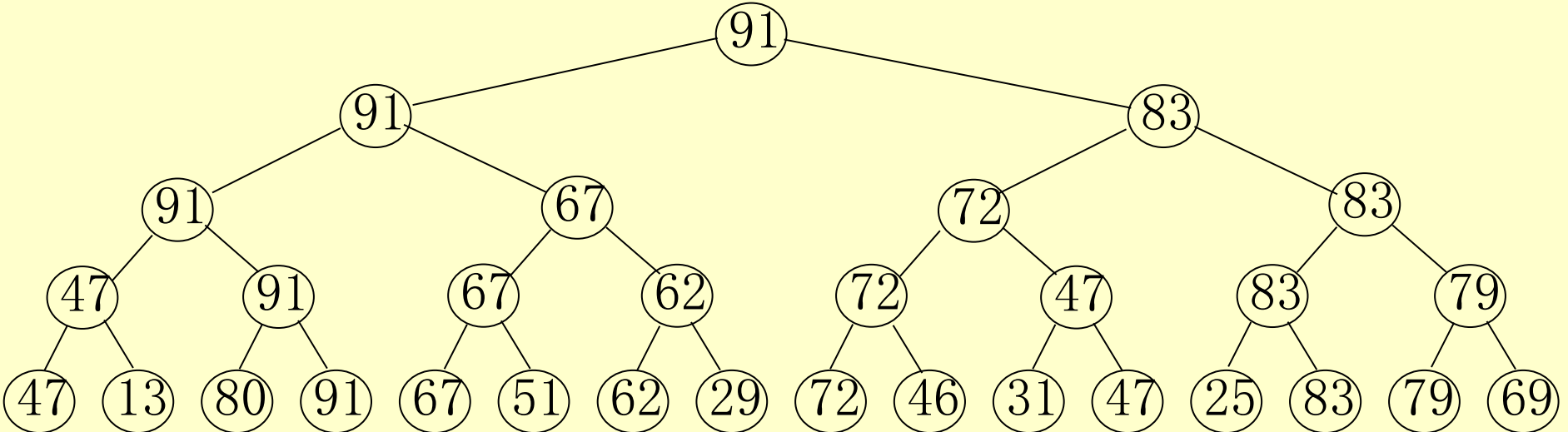
二、树形选择排序

锦标赛排序 胜者树

按照锦标赛的思想进行，将 n 个参赛的选手看成完全二叉树的叶结点，则该完全二叉树有 $2n-2$ 或 $2n-1$ 个结点。

- 首先，两两进行比赛(在树中是兄弟的进行，否则轮空，直接进入下一轮)，胜出的兄弟间再两两比较，直到产生第一名；
- 将作为第一名的结点看成最差的，并从该结点开始，沿该结点到根路径上，依次进行各分枝结点子女间的比较，胜出的是第二名。因为和他比赛的均是刚刚输给第一名的选手。
- 如此，继续进行下去，直到所有选手的名次排定。





■ 算法评价

➤ 时间复杂度

第1次比较需要 $n-1$ 次，其他均为 $\log_2 n$

$O(n * \log_2 n)$

➤ 空间复杂度

$2n-1$ 个结点来存放胜者树

➤ 缺点

- 1、与“ ∞ ”或0比较多余；
- 2、辅助空间使用多。

堆排序对其进行了改进

三、堆排序 p279

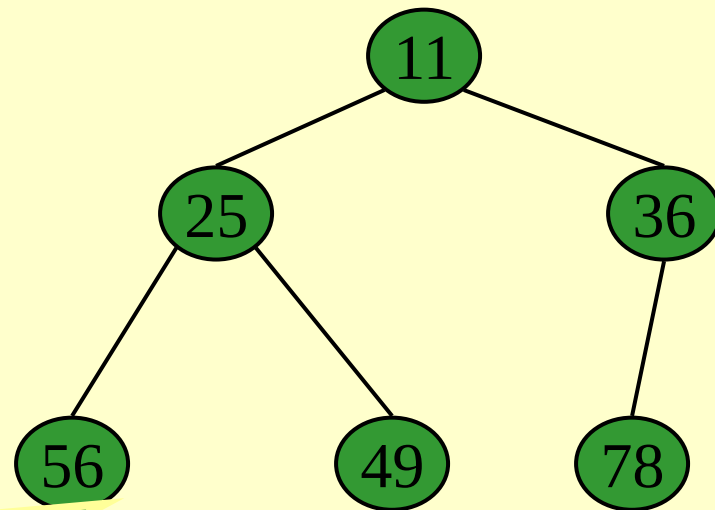
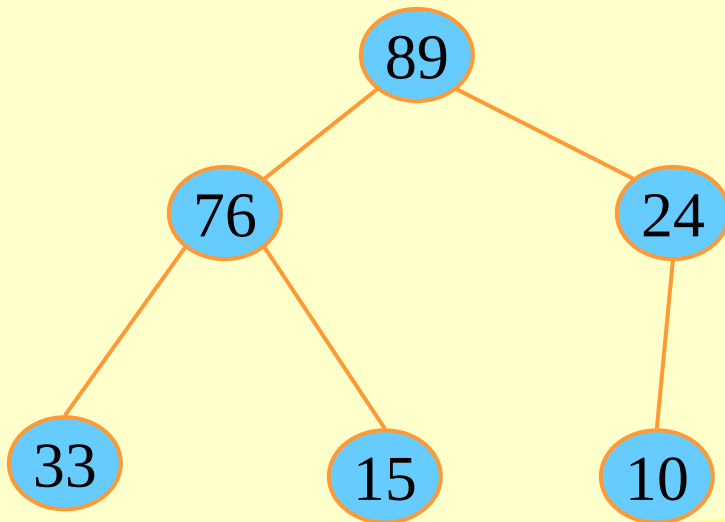
堆：n个元素的序列($k_1, k_2, \dots, k_n$)，当且仅当满足下列关系时，称之为堆。

$$\begin{cases} k_i \leq k_{2i} \\ k_i \leq k_{2i+1} \end{cases} \quad \text{或} \quad \begin{cases} k_i \geq k_{2i} \\ k_i \geq k_{2i+1} \end{cases} \quad (i=1, 2, \dots, \lfloor n/2 \rfloor)$$

? (i=0, 1, \dots)

(89, 76, 24, 33, 15, 10)

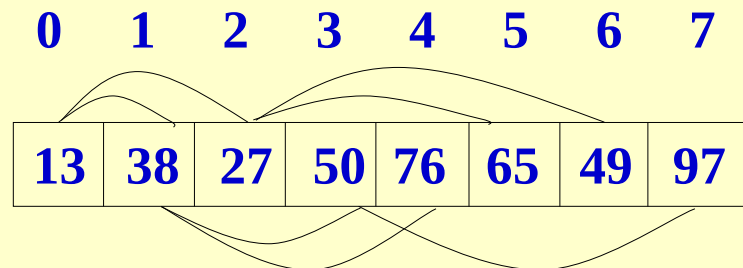
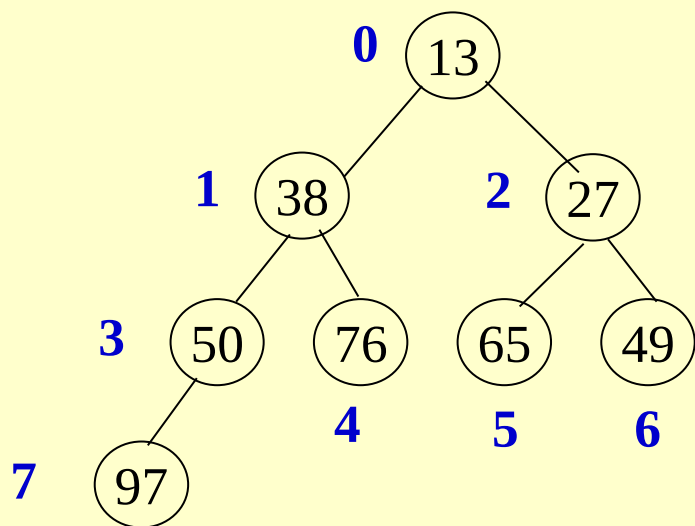
(11, 25, 36, 56, 49, 78)



可将堆序列看成完全二叉树，则堆顶元素（完全二叉树的根）必为序列中n个元素的最小值或最大值

存储及结点计算

为什么用顺序表按层存储？



结点i

- $\text{leftchild}(i) =$
- $\text{rightchild}(i) =$
- $\text{parent}(i) =$
- 非叶子结点数 =

— 堆排序：

- 将无序序列建成一个堆，得到关键字最小（或最大）的记录；
- 输出堆顶的最小（大）值后，使剩余的 $n-1$ 个元素重建成一个堆，则可得到 n 个元素的次小值；
- 直到得到一个有序序列。

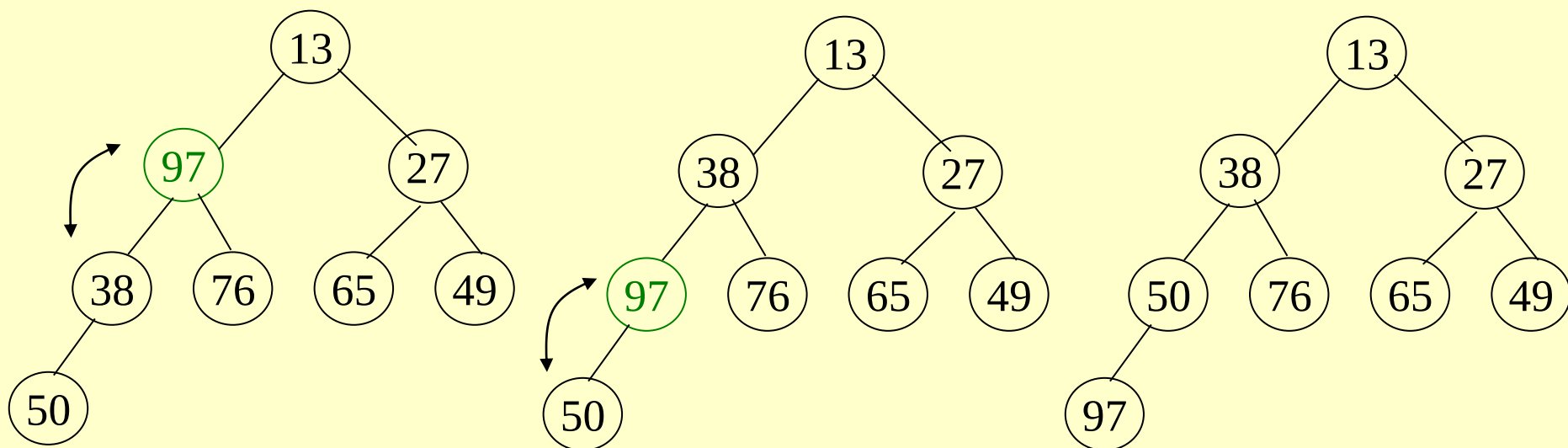
— 堆排序需解决的两个问题：

- 如何由一个无序序列建成一个堆？
- 如何在输出堆顶元素之后，调整剩余元素，使之成为一个新的堆？——**筛选**方法
 - **筛选**输出堆顶元素之后，以堆中最后一个元素替代之；
 - 然后将根结点值与左、右子树的根结点值进行比较，并与其中小者（大者，对于大顶堆）进行交换；
 - 重复上述操作，直至叶子结点，将得到新的堆，从堆顶至叶子的调整过程为“筛选”。

筛选

小顶堆：选择子结点较小者且其小于父结点，与父结点交换。

大顶堆：选择子结点较大者且其大于父结点，与父结点交换。




```
void adjustHeap(int L[],int s, int m )
```

```
{//调整（筛选）L使L[s...m]成为一个大顶堆，与哪些结点有关？
```

```
    int j;
```

```
    int temp = L[s]; //暂存L[s]值到temp
```

```
    for (j = s * 2 + 1; j < m; j = j * 2 + 1) //s指向父结点，j指向子结点
```

```
    {
```

```
        //如果右边值大于左边值，指向右边
```

```
        if (_____) { _____; } //改变j
```

```
        //如果子结点大于父结点，将子结点值赋给父结点,并以新的  
        子结点作为父节点（不用进行交换）
```

```
        if (_____)
```

```
        { _____; _____; }
```

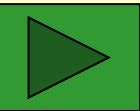
```
        else
```

```
            break;
```

```
    }
```

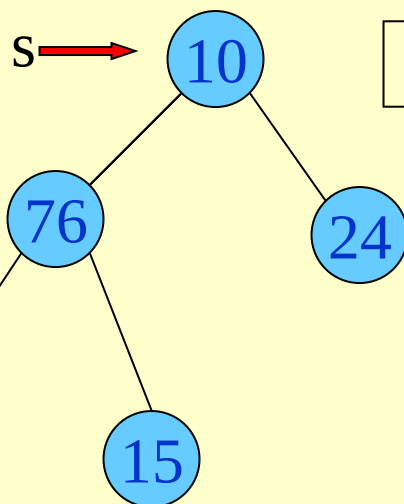
```
    L[s] = temp; //将暂存的值赋值到最后的位置
```

```
}
```



```
void adjustHeap(int L[],int s, int m )
//调整（筛选）L使L[s....m]成为一
个大顶堆
```

由于原根结点
已经保存，所以
实现时一般
不交换



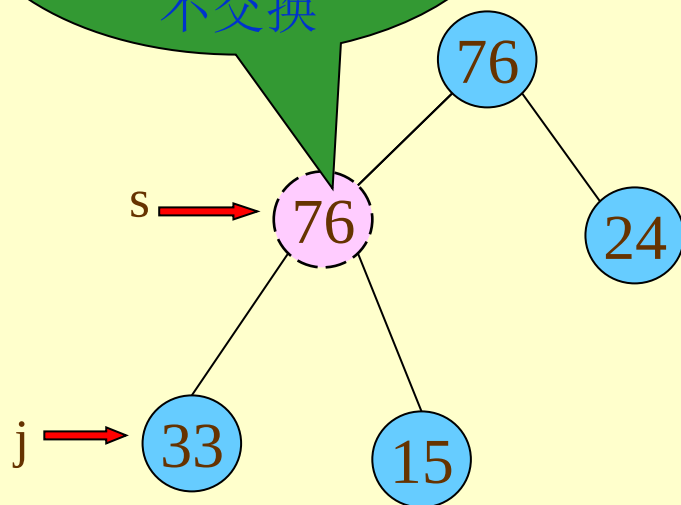
10	76	24	33	15
----	----	----	----	----

0 1 2 3 4

$L[s] = 10$

$m = 5$

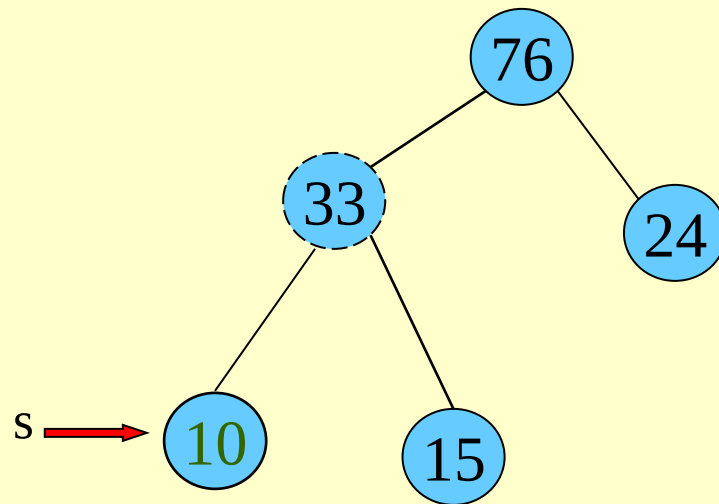
$s = 0$



76	76	24	33	15
----	----	----	----	----

$j = 2*j + 1 = 3$

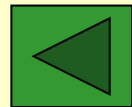
第一次



76	33	24	10	15
----	----	----	----	----

$j = 2*j + 1 = 7 > m$

第二次

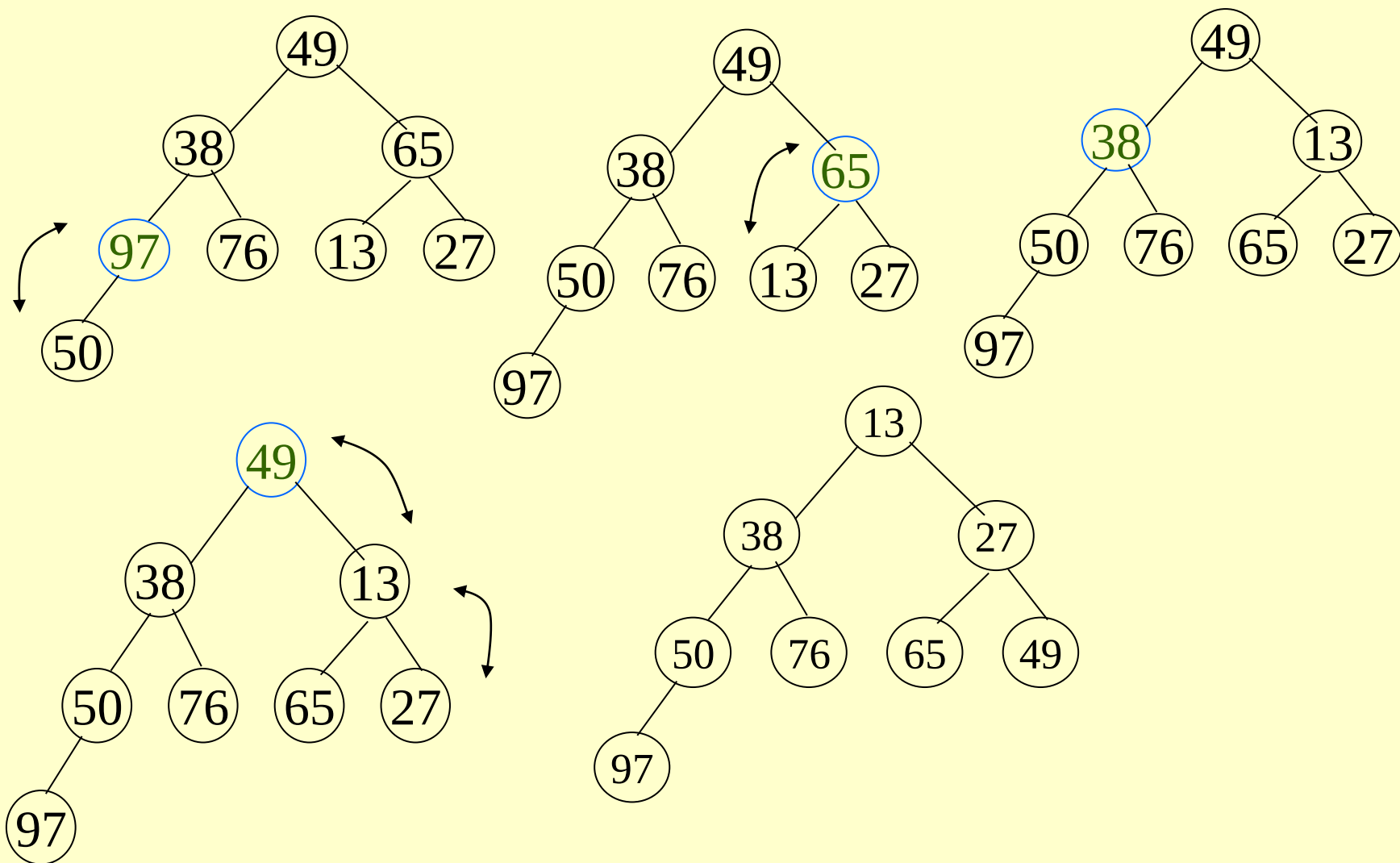


无序序列建成一个堆

- 筛选过程：从无序序列的第 $\lfloor n/2 \rfloor$ 个元素（即此无序序列对应的完全二叉树的最后一个非终端结点）起，至第一个元素止，进行反复筛选；
- 需要筛选结点：逆序，从 $n/2$ 到0；
- 每次筛选范围：为 i 到 n 。

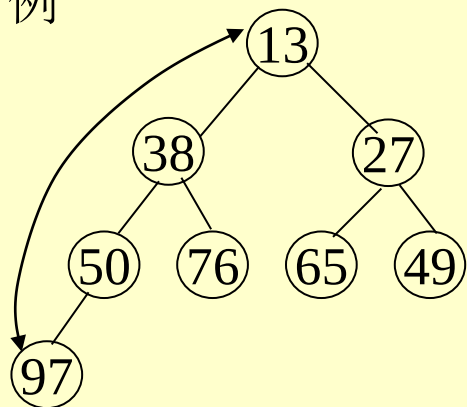
含8个元素的无序序列

(49, 38, 65, 97, 76, 13, 27, 50)



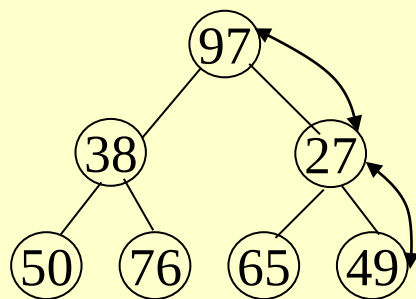
堆排序

例



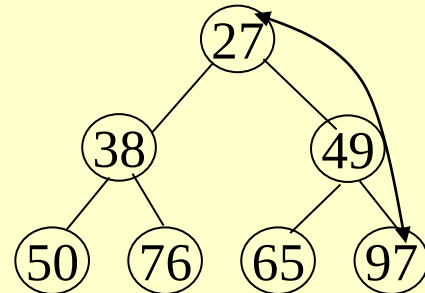
13

输出: 13



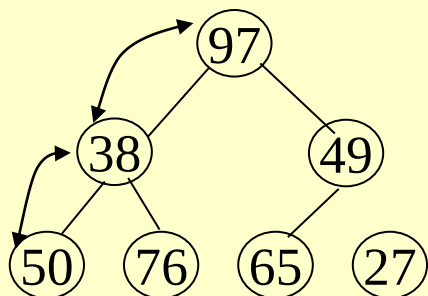
13

输出: 13



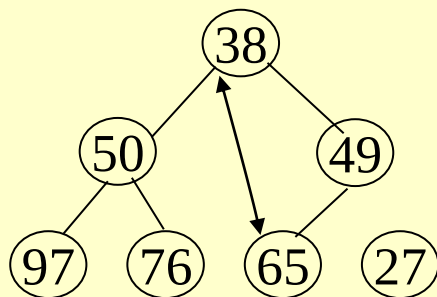
13

输出: 13 27



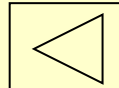
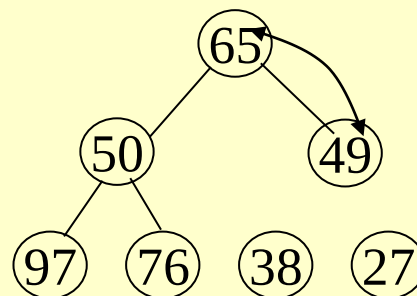
13

输出: 13 27

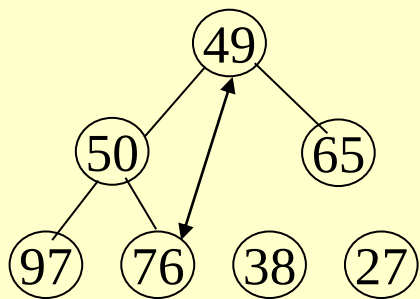


13

输出: 13 27 38

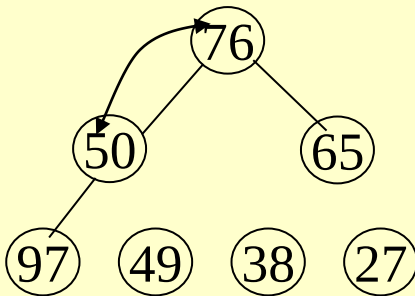


堆排序



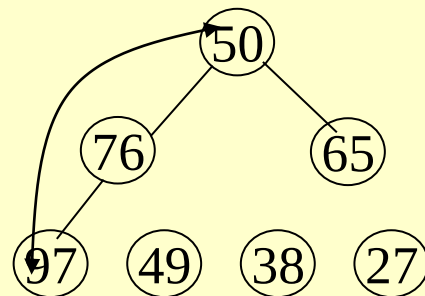
13

输出: 13 27 38



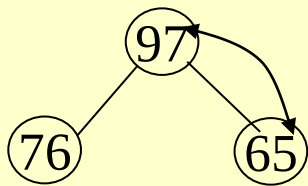
13

输出: 13 27 38 49



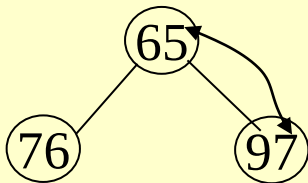
13

输出: 13 27 38 49



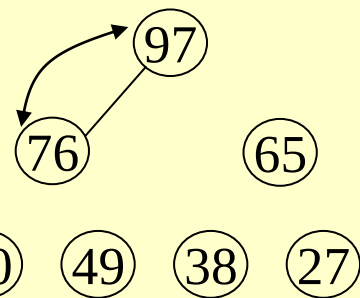
13

输出: 13 27 38 49 50



13

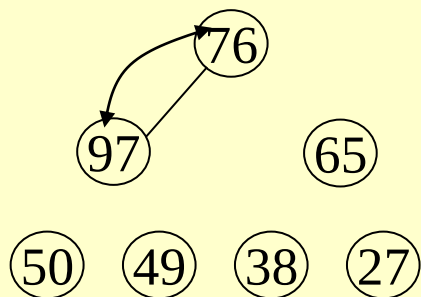
输出: 13 27 38 49 50



13

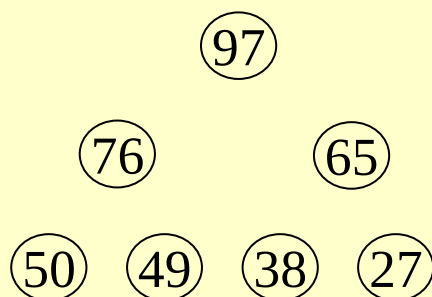
输出: 13 27 38 49 50 65

堆排序



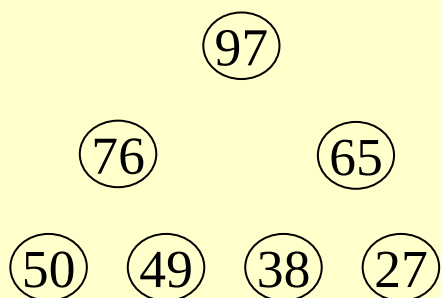
⑬

输出: 13 27 38 49 50 65



⑬

输出: 13 27 38 49 50 65 76



⑬

输出: 13 27 38 49 50 65 76 97

堆排序算法

```
void HeapSort(int L[], int nums)
```

```
{
```

```
    int i,j,temp;
```

```
    //构建大顶堆，需要筛选的结点从nums/2减到0，每次范围为nums/2到nums
```

```
    for (i = nums / 2 ; i >= 0; i--)
```

```
    {    adjustHeap(_____); }
```

```
    printf("构建后的堆结构为: \n");
```

```
    for (i = 0; i<nums; i++) printf("%d ", L[i]); printf("\n");
```

```
    //2.调整堆结构+交换堆顶元素与末尾元素
```

```
    printf("堆排序结果为:\n");
```

```
    for (j = nums - 1; j>0; j--)
```

```
    {
```

```
        printf("%d ", L[0]);
```

```
        //堆顶元素和末尾元素进行交换
```

```
        _____; _____; _____;
```

```
        adjustHeap(_____); //重新对堆进行调整
```

```
    }
```

```
    printf("%d \n", L[0]);
```

```
}
```


- 算法评价

- 时间复杂度：最坏情况下 $T(n)=O(n\log n)$
- 空间复杂度： $S(n)=O(1)$

Experiment 10-3 Heap Sorting

27、【2011统考真题】为实现快速排序算法，待排序序列宜采用的存储方式是（）。

A、顺序存储 B、散列存储 C、链式存储 D、索引存储

28、用基本排序方法对线性表{25,84,21,47,15,27,68,35,20}进行排序时，元素变化情况如下

25, 84, 21, 47, 15, 27, 68, 35, 20

20, 15, 21, 25, 47, 27, 68, 35, 84

15, 20, 21, 25, 35, 27, 47, 68, 84

15, 20, 21, 25, 27, 35, 47, 68, 84

则所采用的排序方法是（）。

A、选择排序 B、插入排序 C、2路归并排序 D、快速排序

29、下列序列中，（）可能是执行第一趟快速排序后所得到的序列。

I. 68, 11, 18, 69, 23, 93, 73 II. 68, 11, 69, 23, 18, 93, 73

III. 93, 73, 68, 11, 69, 23, 18 IV. 68,11,69,23,18,73,93

A、I、IV B、II、III C、III、IV D、只有IV

30、【2014统考真题】下列选项中，不可能是快速排序第2趟排序结果的是（）。

A、2,3,5,4,6,7,9 B、2,7,5,6,4,3,9 C、3,2,5,4,7,6,9 D、4,2,3,5,7,6,9

31、【2010统考真题】采用递归方式对顺序表进行快速排序。下列关于递归次数的叙述中正确是（）

A、递归次数与初始数据的排列次序无关

B、每次划分后，先处理较长的分区可以减少递归次数

C、每次划分后，先处理较短的分区可以减少递归次数

D、递归次数与每次划分得到的分区的处理顺序无关

32、【2019统考真题】排序过程中，对沿未确定最终位置的所有元素进行一遍处理称为一趟，下列序列中不可能是快速排序第二趟结果的是（）。

A、5,2,16,12,28,60,32,72 B、2,16,5,28,12,60,32,72 C、2,12,16,5,28,32,72,60 D、5,2,12,28,16,32,72,60

33、若只想得到1000个元素组成的序列中第10个最小元素之前的部分排序的序列，用（）方法最快。

A、冒泡排序 B、快速排序 C、希尔排序 D、堆排序

34、下列（）是一个堆。

A、19,75,34,26,97,56 B、97,26,34,75,19,56

C、19,56,26,97,34,75 D、19,34,26,97,56,75

35、在含有n个关键字的小根堆中，关键字最大的记录有可能存储在（）位置。

A、 $n/2$ B、 $n/2+2$ C、1 D、 $n/2-1$

36、【2009统考真题】已知关键字序列5,8,12,19,28,20,15,22是小根堆，插入关键字3，调整后得到的极小堆是（）。

A、3,5,12,8,28,20,15,22,19 B、3,5,12,19,20,15,22,8,28

C、3,8,12,5,20,15,22,28,19 D、3,12,5,8,28,20,15,22,19

37、【2011统考真题】已知关键字序列25,13,10,12,9是大根堆，在序列尾部插入新元素18，将其再调整为大根堆，调整过程中元素之间进行的比较次数是（）。

A、1 B、2 C、4 D、5

38、下列4种排序方法中，排序过程中的比较次数与序列初始状态无关的是（）。

A、选择排序 B、快速排序 C、插入排序 D、冒泡排序

39、【2015统考真题】已知小根堆为8,15,10,21,34,16,12，删除关键字8之后需重建堆，在此过程中，关键字之间的比较次数是（）。

A、1 B、2 C、3 D、4